Tik-Booth Developer Guide

Audience: Developers new to the repository, including junior engineers **Repository snapshot:** November 3, 2026 **Purpose:** Explain how the application is organized and how its main user and data flows work, so a developer can make changes without breaking important guarantees.

> This guide intentionally excludes CSS, styling implementation, and visual design details. It describes application behavior, architecture, data ownership, security boundaries, and development practices.

Contents

1. Start here
2. What the application does
3. Technology and request architecture
4. Repository map
5. Routes and application surfaces
6. Database concepts
7. Important end-to-end flows
8. Security and correctness rules
9. Configuration and local development
10. How to make a change safely
11. How to scale the application
12. Current project status and documentation notes
13. Glossary

1. Start here

If you are new to the codebase, read these files in this order:

1. `README.md` for the short product summary.
2. `context/project-overview.md` for user journeys and non-negotiable business rules.
3. This guide for the current repository layout and how the parts fit together.
4. `context/architecture.md` for deeper data-flow and database notes. Some of its example paths describe an earlier planned layout; the current layout is explained in Section 4.
5. `context/code-standards.md` for TypeScript, server/client, API, payment, and error-handling rules.
6. `context/library-docs.md` before changing code that uses Supabase, Paystack, PostHog, ticket QR codes, image uploads, or other documented dependencies.

7. `context/build-plan.md` and `context/progress-tracker.md` to understand the planned phases and what still needs verification.

The code is the final authority when documentation and implementation disagree. In particular, this repository currently puts `app`, `actions`, `components`, `lib`, `types`, and `utils` at the repository root. It does not use the older `src/` layout shown in parts of `context/architecture.md`.

2. What the application does

Tik-Booth is a web event-ticketing platform. Event organizers publish in-person or online events, define ticket tiers, and receive ticket-sale payouts through Paystack. Attendees discover events, reserve and buy tickets, and present a QR-backed ticket at the door or use a ticket to access an online event.

The app is built around two high-risk guarantees:

- **A ticket tier must not oversell.** The database atomically reserves inventory before payment begins.
- **A ticket must not admit two people.** A signed QR token is checked server-side, and check-in changes the ticket state atomically.

The main user roles are:

- **Attendee:** can discover events, buy or claim tickets, see their tickets, and use ticket-protected online event access.
- **Organizer:** starts as an attendee, then completes organizer onboarding to publish events, configure ticket tiers and payouts, see sales information, and manage gate access.
- **Door staff:** does not need a Tik-Booth account. A gate-specific access link grants narrowly scoped check-in access and can be revoked.

At the current product scope, the application supports NGN, Paystack, online-only check-in, and single-owner event management. Resale, ticket transfer, refunds, offline scanning, assigned seating, and multi-currency checkout are intentionally out of scope.

3. Technology and request architecture

Main technologies

Area	Technology	Responsibility
Web application	Next.js App Router 16	Server-rendered pages, client interactivity, route handlers, and Server Actions
Language	TypeScript strict mode	Application types and compile-time checks
Data and authentication	Supabase	PostgreSQL, Auth, Storage, Row Level Security, and database functions
Payments	Paystack REST API	Payment initialization, verification, and split payouts behind a provider interface

Area	Technology	Responsibility
Client-refetched server data	TanStack Query	Refreshing data such as the event feed
Abuse protection	Upstash Redis and rate limiting	Limits checkout attempts and door scans
Analytics	PostHog	Approved product events and organizer funnel metrics
Ticket code	Node crypto and `qrcode`	HMAC-signed ticket tokens and their QR images
3D ticket object	Three.js and React Three Fiber	Real extruded 3D ticket meshes in the ticket wallet and organizer previews
Image processing	Sharp	Validates and optimizes organizer/event/ticket image uploads
Deployment	Vercel and Supabase	Application hosting and managed data services

The package manifest is the source of truth for installed dependencies. The repository currently does not include Zustand, despite older architecture documentation describing it as planned state management. Do not add a dependency just because it appears in an older design note.

How a request moves through the app

1. A browser requests an App Router page under `app/`.
2. A Server Component can authenticate the request, load data through the server Supabase client, and render the initial page.
3. A Client Component is used only when a page needs browser behavior such as user interaction, camera access, local state, or an installed client-only library.
4. A page may call a route handler under `app/api/` for an HTTP operation or a Server Action under `actions/` for an authenticated UI mutation.
5. The route/action delegates domain logic to a function in `lib/`, or directly uses the appropriate Supabase client for straightforward page data.
6. Supabase Row Level Security (RLS) and database functions enforce the final data boundary and high-integrity transitions.

The server Supabase client reads the current request's cookies. The browser Supabase client is for browser-only tasks. The service-role client bypasses RLS and must remain inside trusted server code.

Server and client responsibility

- Pages and layouts are Server Components by default.
- Keep authentication checks, protected data loading, ownership checks, and secrets on the server.
- Add `"use client"` only where browser APIs, local interaction, or client libraries are required.
- A Client Component may receive display-ready data from its server parent. It should not become a back door for privileged database or payment operations.
- Server Actions are for UI-triggered writes. HTTP APIs and external callbacks belong in route handlers.

4. Repository map

The following is the **current** repository structure. The list focuses on application code and operational documentation; it omits styling files by request.

```
/
├── actions/
│   ├── door-staff.ts
│   └── onboarding.ts
├── app/
│   ├── api/
│   │   ├── checkin/scan/route.ts
│   │   ├── checkout/start/route.ts
│   │   ├── checkout/status/route.ts
│   │   ├── events/discover/route.ts
│   │   ├── jobs/expire-reservations/route.ts
│   │   ├── onboarding/images/route.ts
│   │   └── webhooks/paystack/route.ts
│   ├── auth/callback/route.ts
│   ├── checkout/[event_id]/page.tsx
│   ├── dashboard/
│   ├── e/[event_id]/page.tsx
│   ├── login/
│   ├── my-tickets/
│   ├── onboarding/
│   ├── profile/page.tsx
│   ├── scan/[event_id]/page.tsx
│   ├── page.tsx
│   └── layout.tsx
├── components/
│   ├── checkin/
│   ├── checkout/
│   ├── dashboard/
│   ├── discover/
│   ├── door-staff/
│   ├── events/
│   ├── layout/
│   ├── onboarding/
│   ├── profile/
│   ├── providers/
│   └── tickets/
├── context/
│   ├── architecture.md
│   ├── build-plan.md
│   ├── code-standards.md
│   ├── library-docs.md
│   ├── progress-tracker.md
│   ├── project-overview.md
│   ├── ui-registry.md
│   ├── ui-rules.md
│   └── ui-tokens.md
├── lib/
├── supabase/
│   ├── migrations/
│   ├── schema.sql
│   └── config.toml
├── types/
├── utils/
├── instrumentation-client.ts
├── instrumentation.ts
├── proxy.ts
├── package.json
├── tsconfig.json
└── vercel.json
```

What belongs in each directory

Path	Put this kind of code here
`app/`	Route pages, layouts, and HTTP handlers. Pages coordinate data loading and render components; route handlers validate an HTTP request and orchestrate work.
`app/api/`	Public or authenticated HTTP endpoints, incoming payment webhooks, and scheduled jobs. Keep domain work in `lib/` rather than growing route handlers into business-logic modules.
`actions/`	Server Actions triggered by the UI. Current files implement organizer onboarding and gate-link mutations.
`components/`	Page and feature components grouped by domain. Keep Supabase ownership checks and payment secrets out of components.
`lib/`	Reusable domain logic, shared constants, provider abstractions, and server-only services such as checkout, QR signing, event reads, and gate validation.
`utils/`	Third-party client setup and narrow integration utilities. Examples include Supabase clients and server-side PostHog capture.
`types/`	Generated Supabase database types and shared application types such as public event data.
`supabase/migrations/`	Ordered, versioned schema and database-function changes. Add a migration for a schema change rather than editing only a local database.
`supabase/schema.sql`	A readable schema snapshot/reference. Migrations are the chronological change history.
`context/`	Product scope, architecture, coding standards, library patterns, and progress notes. Keep the docs aligned when a decision or implementation changes.
`proxy.ts`	Request-level Supabase session refresh and the protected-route matcher. This is not a replacement for authorization checks in the data operation.
`instrumentation-client.ts`	Browser PostHog startup and identity lifecycle.
`instrumentation.ts`	Server instrumentation and structured log integration.

Important current files

- `lib/checkout.ts` owns checkout orchestration and calls the atomic database operations.
- `lib/payment-provider.ts` defines the processor-neutral payment contract; `lib/paystack-provider.ts` implements it.
- `lib/events.ts` reads published event data for server-rendered pages and the discovery endpoint.
- `lib/qr.ts` creates and verifies server-side ticket credentials. It is marked server-only.
- `lib/door-staff.ts` validates gate credentials and calls narrow scan-related database functions.
- `lib/rate-limit.ts` defines rate-limiting behavior for sensitive endpoints.
- `lib/constants.ts` keeps business limits such as commission, reservation time, image size, and ticket-per-order maximum in one place.
- `utils/supabase/client.ts`, `server.ts`, and `admin.ts` provide separate browser, cookie-aware server, and trusted server-admin clients.

- `types/database.types.ts` represents the generated database API. `types/events.ts` provides application-level public event shapes.
- `components/tickets/` contains the ticket wallet component, the React Three Fiber renderer, and the accessible fallback.
- `components/onboarding/` contains wizard steps and reusable onboarding interactions; the Server Action layer persists the data.
- `app/preview/online-join/page.tsx` is a development preview, not a production source of online-event access.

5. Routes and application surfaces

User-facing pages

URL	Purpose	Access model
`/`	Discover events and initialize the public feed	Public
`/login`	Email/password and Google sign-in or sign-up	Public
`/auth/callback`	Exchange the Supabase OAuth code for a session	OAuth callback
`/e/[event_id]`	Published event detail and ticket selection	Public
`/checkout/[event_id]`	Checkout progress and payment result/status	Signed-in buyer for protected status
`/my-tickets`	Buyer's ticket wallet	Signed-in user
`/my-tickets/[ticket_id]/join`	Online event access; the server decides whether access is unlocked	Signed-in owner of an eligible ticket
`/profile`	Account settings, organizer entry point, appearance preference	Signed-in user
`/onboarding/profile`	Organizer identity/profile	Signed-in organizer
`/onboarding/payout`	Paystack payout setup	Signed-in organizer
`/onboarding/event`	Create or resume an event draft	Signed-in organizer
`/onboarding/tiers`	Configure ticket tiers and card data	Signed-in organizer
`/onboarding/publish`	Validate and publish the selected draft	Signed-in organizer
`/dashboard`	Organizer overview and owned events	Signed-in organizer
`/dashboard/[event_id]`	Event-specific sales/operations information	Event owner
`/dashboard/[event_id]/door-staff`	Gate link management	Event owner
`/scan/[event_id]`	Door staff entry and live scanner	Valid gate link; no user account

`proxy.ts` currently applies an authentication gate to dashboard, ticket wallet, onboarding, and profile routes. Individual pages/actions still must verify ownership and authorization. The scanner is deliberately not in that signed-in matcher: its credential is a revocable per-event gate token.

HTTP endpoints

Method and path	Purpose
`GET /api/events/discover`	Return the published event feed.
`POST /api/checkout/start`	Validate checkout input and start a free issuance or paid reservation/payment flow.
`GET /api/checkout/status?reference=...`	Return the authenticated buyer's reservation/payment state.
`POST /api/webhooks/paystack`	Verify a Paystack event and process the payment result idempotently.
`GET /api/jobs/expire-reservations`	Cron-protected job that checks pending payments before it releases expired holds.
`POST /api/checkin/scan`	Validate the gate, rate-limit the gate, verify the QR token, and ask the database to atomically scan.
`POST /api/onboarding/images`	Validate and optimize an authenticated organizer's staged image upload.

Success responses normally use `{ success: true, data }`; failures use `{ success: false, error }` and an appropriate HTTP status. Never return raw provider errors or secrets to the browser.

6. Database concepts

Supabase is both the application's persistent data store and an authorization boundary. RLS policies restrict rows, and carefully scoped Postgres functions perform operations that must be atomic or must reveal sensitive data only under strict conditions.

Core records

Record	Purpose
`profiles`	One account profile per Supabase Auth user. `is_organizer` marks an upgraded account; there is one sign-in model.
`events`	Event metadata and publication state. Event owner is `organizer_id`; events have a format, location, start/end timestamps, and status.
`ticket_types`	Ticket tiers, integer-kobo prices, sale capacity, and organizer-selected ticket-card data.
`reservations`	Temporary buyer-specific inventory holds and payment state. A pending paid reservation is not yet a ticket.
`orders`	Confirmed purchase record, payment reference, total, and platform commission.
`tickets`	Individual attendee credentials, QR token, and lifecycle status (`issued`, `used`, or `void`).

Record	Purpose
`door_staff_access`	Per-event gate identity, gate label, hashed access token, and revocation state.
`processed_webhook_events`	Payment-provider event IDs already processed, used to make retry handling idempotent.
`private.event_online_join_links`	Online meeting URL kept outside client-readable event data.

Events and their tiers are the public discoverable catalog. A reservation is a short-lived claim against inventory. An order records a completed payment. Each ticket is a single admission credential. These are separate concepts; do not treat a redirect back from Paystack as proof that an order or ticket exists.

Database functions and why they matter

Database RPCs (PostgreSQL functions called through Supabase) provide atomicity and privacy where several read/write steps cannot safely be split between application requests:

- `reserve_ticket_inventory` reserves a requested quantity only if the capacity constraint still holds.
- `release_expired_reservation` releases a hold only after the scheduled job has checked provider state and the database confirms it is safe.
- `process_paid_reservation` validates and records payment, order, tickets, and webhook idempotency as one database operation.
- `issue_free_ticket_order` issues free tickets through a restricted server-authorized path.
- `validate_door_staff_access`, `get_door_staff_scan_context`, and `scan_door_staff_ticket` validate gate credentials and provide event-scoped atomic scan behavior.
- `get_online_event_join_access` checks the current user, ticket ownership/status, payment, event, and database time before returning an online URL.
- Onboarding/event/tier RPCs validate ownership and publish state where a multi-record change must be all-or-nothing.

The exact current function definitions and grants are in `supabase/migrations/` and `supabase/schema.sql`. When changing a function signature, update its callers and generated database types as part of the same task.

Row Level Security and storage

RLS is enabled for application tables. Always use both:

1. Application-level user/organizer filters and explicit ownership checks, which give clear errors and reduce accidental broad reads.
2. Database RLS, which remains the authoritative boundary if an application query is incorrect.

Image uploads use a private staging bucket followed by an authenticated server route. That route checks source ownership and target ownership/status, checks the actual image format and size, optimizes the image, stores it in the final purpose-specific bucket, updates the owning record, and cleans up staged/replaced objects. Do not trust a filename or a client-side file-type check as validation.

7. Important end-to-end flows

7.1 Discover and event detail

1. `app/page.tsx` loads the first published event feed using `lib/events.ts`.
2. It passes the initial result to `DiscoverPageClient`, which provides browser-side search/filter interaction and refresh through `usePublishedEvents`.
3. `GET /api/events/discover` reuses the same server-side feed function for refreshes.
4. `app/e/[event_id]/page.tsx` validates the route ID and loads only a published event.
5. The server passes event data and a server timestamp into the detail component. Availability and status should not rely on an untrusted browser clock.

The public event type is defined in `types/events.ts`. Keep it display-safe; do not include private join URLs or organizer payout data in this response.

7.2 Checkout, payment, and ticket issuance

1. The buyer selects a tier and quantity. Client-side checks are useful for usability but are not the security boundary.
2. `POST /api/checkout/start` parses and validates the request. `lib/checkout.ts` validates UUIDs, quantity, authenticated user, event/tier relationship, and event availability.
3. The user is rate-limited. Free orders use a restricted issuance RPC and do not call Paystack.
4. Paid checkout calls `reserve_ticket_inventory`. If inventory is unavailable, the request stops before Paystack. A successful call creates a time-limited reservation, normally 10 minutes.
5. The `PaymentProvider` implementation initializes Paystack. The buyer is sent to Paystack's hosted checkout page.
6. A browser return from Paystack is only a status-check hint. The application queries `GET /api/checkout/status`; the webhook is the source of truth for payment confirmation.
7. Paystack calls `POST /api/webhooks/paystack`. The handler reads the raw request body, verifies the HMAC-SHA512 signature, parses the event through the provider interface, and delegates the paid transition to the database RPC.
8. The database operation records the event ID for idempotency, confirms the reservation, creates the order and issued tickets, and prevents duplicate issuance on retries.
9. Ticket QR credentials are generated server-side with HMAC-SHA256 over a ticket UUID. The QR image is rendered in the client from the already-signed token; the browser never signs a ticket.
10. An authenticated cron job checks old pending reservations with Paystack before releasing them. A pending or successful payment must not lose inventory due to a timer alone.

Money is stored and calculated in integer kobo. The commission and maximum ticket quantity come from `lib/constants.ts`.

7.3 Organizer onboarding and publishing

1. A signed-in user chooses “Become an organizer” from Profile.
2. The onboarding Server Actions authenticate the current user and verify organizer state before writing.

3. The organizer profile step writes account data; optional images go through the staged upload and optimization route.
4. The payout step calls the `PaymentProvider` abstraction. Bank details are sent for Paystack verification/recipient creation and are not stored as the organizer's bank details.
5. Event setup creates a specific owned draft. The draft ID is passed through subsequent steps so resuming one draft cannot accidentally update another.
6. Ticket tier setup validates price/capacity and card configuration. Tier images use the same staged image workflow.
7. The publish operation validates the required organizer, event, payout, and tier records through the database's ownership-checked operation before switching the draft to published.
8. Each step is resumable using `profiles.onboarding_step`; profile and payout setup can be reused for future events.

Do not treat “the user is in the onboarding route” as proof of authorization. Server Actions must obtain the user from Supabase and scope every write to their profile/event.

7.4 Door staff and check-in

1. An event owner creates a gate through a Server Action. The application generates a random bearer token and stores only its SHA-256 hash.
2. The plaintext link is returned once for the organizer to share. It should not be logged, stored for later retrieval, or exposed in analytics.
3. The usher opens `/scan/[event_id]` with the gate token. No Supabase account is created.
4. The server validates the token against the event and checks revocation before rendering the scanner.
5. The browser decodes the ticket QR with the camera and sends only the signed QR token and event ID to the scan endpoint. Video or camera frames are not uploaded.
6. The endpoint validates the gate token and rate limit, verifies the QR signature, then calls the database scan RPC.
7. The database changes a ticket from `issued` to `used` only once. A concurrent or repeated scan returns the already-used outcome, including the original scan context.
8. The scanner is online-only. A connectivity failure must show “No connection”; never accept or queue an unverified scan.

7.5 Online event join access

The join URL is kept in a private schema, not exposed by the public event record. The join page authenticates the user and calls `get_online_event_join_access` with the ticket ID. The database verifies ticket ownership and eligibility and uses database time to withhold the link until 30 minutes before event start. The page renders a countdown from server-returned timestamps; a hidden button or client-only guard would not protect the URL.

7.6 Organizer analytics

Supabase stores the operational truth: paid orders, tickets, scans, and gate labels. PostHog stores product funnel events. The dashboard fetches metrics on the server and scopes event IDs through organizer-owned data before querying PostHog. Do not expose a shared raw analytics query endpoint to the browser.

Approved custom product events are listed in `context/code-standards.md`. Payment-confirmed and check-in outcome events must be fired from the server. Never include passwords, email addresses, raw search strings, bearer

tokens, or other unnecessary sensitive data as analytics properties.

8. Security and correctness rules

These are not optional implementation preferences. They protect payment, inventory, ticket access, and attendee privacy.

1. **Never change `ticket_types.quantity_sold` with a normal client update.** Use the atomic reservation/release database operations.
2. **Never issue a paid ticket based only on the browser redirect.** Only a signature-verified provider webhook and its idempotent database operation confirm the payment.
3. **Verify Paystack using the raw body and HMAC-SHA512.** Parsing/re-serializing first changes the signed data. Follow the provider implementation.
4. **Never store a floating-point money amount.** Use integer kobo; convert only for display/analytics when required.
5. **Never place a raw ticket ID in the QR code.** QR payloads are server-generated signed tokens and are verified before ticket state is queried.
6. **Never check ticket state before validating the gate credential.** Invalid or revoked access must not reveal ticket information.
7. **Never use the service-role key in browser code.** It bypasses RLS. Keep admin client imports server-only and narrowly scoped.
8. **Always scope reads and writes by the current user or organizer.** RLS is the safety net, not an excuse to omit application checks.
9. **Do not reveal the online join URL early.** Keep it private and let the database enforce ownership and unlock time.
10. **Treat callback input as untrusted.** Parse request bodies, validate IDs/ranges/allowed values, and reject unsafe redirect destinations.
11. **Rate-limit sensitive actions and fail closed** when the configured limiter is unavailable; do not silently bypass protection.
12. **Log failures with a stable context prefix and return user-safe messages.** Do not expose provider/database internals.
13. **Do not add unapproved analytics events or sensitive properties.** Update the event catalog and review privacy impact first.
14. **No offline scan fallback.** An offline device cannot know whether another gate already accepted the same ticket.
15. **No resale, transfer, or refund flows** without an explicit product-scope decision and corresponding data/operational review.

9. Configuration and local development

Environment variables

Names below describe configuration only. Their secret values must never be copied into source code, examples, documentation, logs, or chat.

Variable	Use	Exposure
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL	Browser and server
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase public/anonymous key	Browser and server
`NEXT_PUBLIC_SITE_URL`	Trusted base URL for payment callbacks	Browser-visible setting
`SUPABASE_SERVICE_ROLE_KEY`	Restricted trusted server operations	Server only
`PAYSTACK_SECRET_KEY`	Paystack API operations	Server only
`PAYSTACK_WEBHOOK_SECRET`	Verify Paystack webhook signatures	Server only
`QR_TOKEN_SECRET`	Sign/verify ticket QR tokens	Server only
`UPSTASH_REDIS_REST_URL`	Rate-limit Redis endpoint	Server configuration
`UPSTASH_REDIS_REST_TOKEN`	Rate-limit Redis authentication	Server only
`CRON_SECRET`	Authenticate the reservation-expiry job	Server only
`NEXT_PUBLIC_POSTHOG_PROJECT_TOKEN`	Browser/server PostHog project key	Public project token
`NEXT_PUBLIC_POSTHOG_HOST`	PostHog ingestion host	Browser and server
`POSTHOG_PROJECT_API_KEY`	Server-side event capture integration	Server only
`POSTHOG_API_HOST`	Private PostHog query host	Server only
`POSTHOG_PROJECT_ID`	Private PostHog query project	Server only
`POSTHOG_PERSONAL_API_KEY`	Private funnel query access	Server only

The `.env.local` file is local configuration and must remain private. Any variable containing credentials, service-role access, signing material, or personal API access must not have a `NEXT_PUBLIC_` prefix.

Commands

The repository package manager is Bun (`package.json` specifies Bun 1.3.8). From the repository root:

```
bun install
bun run dev
bun run lint
bunx tsc --noEmit
bun run build
bun run start
```

There is no dedicated test script in the current `package.json`. Run the narrowest applicable checks, then the production build for changes affecting routes, server/client boundaries, or shared types. Browser-level verification is important for camera, payment callback, theme persistence, and ticket rendering behavior.

Supabase schema work

1. Make schema changes through a new ordered migration in `supabase/migrations/`.

2. Include RLS, grants, function ownership/security mode, and constraints in the migration.
3. Test the positive and negative access cases, not only the intended happy path.
4. Regenerate or reconcile `types/database.types.ts` against the changed schema.
5. Update `supabase/schema.sql` if the project uses it as a current schema snapshot.
6. Record migration application and validation status in `context/progress-tracker.md`.

Never apply a migration twice just because it appears in a local file list. Check the linked database migration history first.

10. How to make a change safely

For a new page or user journey

1. Find the route under `app/` and confirm whether it should be public, signed-in, owner-only, or gate-token-only.
2. Find related components in the matching `components/` feature folder.
3. Keep data loading in the Server Component when possible. Pass only the fields that the component needs.
4. If the page needs client refresh, use the existing React Query patterns; do not duplicate persistent server data in unrelated client state.
5. Consider loading, empty, invalid-ID, unauthenticated, not-found, and server-error states.
6. Verify that navigation and auth redirect behavior still work.

For a mutation

1. Decide whether it is an HTTP endpoint (`app/api/`) or a UI Server Action (`actions/`).
2. Validate every external/client-provided field on the server.
3. Call `supabase.auth.getUser()` for trusted identity, then check ownership explicitly.
4. Use the server client for normal user-scoped operations; use the admin client only when a trusted server-only flow requires it.
5. Put repeated or security-sensitive domain behavior in `lib/`.
6. Use a Postgres function for atomic transitions and restricted data release.
7. Revalidate affected paths after successful Server Action writes.
8. Return clear, typed success/failure states and log unexpected failures without exposing internals.

For payment, inventory, ticket, or scanner changes

Pause before editing. Trace all callers, database functions, webhook retries, scheduled jobs, ticket wallet reads, and dashboard analytics. These are coupled parts of one transaction lifecycle. Add tests or database checks for concurrency, retries, invalid credentials, and partial failures. Review `context/library-docs.md` before changing the integrations.

For new external dependencies

Check whether the platform or an installed package already provides the needed capability. Review `context/library-docs.md` and the package's current official guidance. Add a dependency only when justified, update

the package manifest and library guidance, and document why the dependency belongs in the architecture.

11. How to scale the application

Scale by preserving existing boundaries instead of putting more logic in page components.

Adding a payment provider

Implement `PaymentProvider` and make provider selection explicit in server configuration. Keep transaction initialization, webhook verification/parsing, transaction verification, and payout-recipient creation behind the contract. Add provider-specific tests and verify commission semantics before production use.

Adding a new business workflow

Model its state transitions and ownership first. Keep records in Supabase; use constraints and RPCs for concurrent/atomic transitions. Add the page/route and feature components separately. Update generated types, architecture notes, approved analytics events, and progress tracking as relevant.

Increasing traffic

- Keep public event reads selective and paginate/filter at the database/API rather than shipping unbounded records.
- Keep inventory-bearing data uncached or use a deliberately short freshness policy with server validation at checkout.
- Keep Redis limiters available and observable; do not make high-risk endpoints fail open.
- Keep background maintenance bounded and concurrency-limited as the expiry route currently does.
- Index actual query patterns and validate changes with query plans before adding speculative indexes.
- Keep PostHog funnel queries organizer-scoped and bounded to the requested reporting window.
- Monitor server errors, payment webhook retries, expired reservations, image processing, and scanner availability.

Adding a new domain folder

Create a domain folder under `components/` for its UI and place reusable domain operations in `lib/`. Add routes only for URL-addressable pages and `app/api/` only for HTTP interfaces. Avoid barrel exports outside the UI primitive directory; use the repository's `@/` import alias.

Maintaining docs

When a route, business invariant, dependency, database object, environment variable, or ownership rule changes, update the nearest project document and `context/progress-tracker.md`. Keep documentation tied to the real folder layout. Do not copy planned architecture diagrams as if they were current without checking the code.

12. Current project status and documentation notes

This guide records the repository and its progress notes as of November 3, 2026. A feature marked implemented is not necessarily production-verified.

The progress tracker reports that:

- The Phase 6 online event access migration is applied and its privacy/access-boundary checks passed.
- No complete online event plus issued-ticket fixture was available for an end-to-end locked/unlocked access test.
- Event detail real-data behavior still awaits verification against a suitable published event.
- Paystack sandbox/webhook verification remains outstanding.
- PostHog private-query settings are still required for live organizer funnel analytics.
- An end-to-end check-in flow and visual sign-off for some organizer/dashboard states remain outstanding.
- The development-only online join preview uses mock data and is not the real join-access flow.

There are documentation differences to keep in mind:

- Some of `context/architecture.md` shows the planned `src/` layout. The working repository uses root-level directories.
- `README.md` examples mention npm and a type output path under `src/`. The package manifest specifies Bun, and the current generated type file is `types/database.types.ts`.
- Some older architecture text lists Zustand and files that do not exist in the current package/source tree. Verify actual installed dependencies and current files before relying on such examples.
- `context/project-scope.md` is referenced by other documents but is not present at the repository root or under `context/` in this snapshot. Treat `context/project-overview.md`, `build-plan.md`, `code-standards.md`, and the progress tracker as the available project decisions, and ask for scope confirmation before adding a feature explicitly marked out of scope.

These are maintenance notes, not invitations to perform broad cleanup while implementing an unrelated feature.

13. Glossary

Term	Plain-language meaning
App Router	Next.js routing model where folders and special files under `app/` define pages and endpoints.
Server Component	A React component rendered on the server; useful for private data loading and reducing browser-side access.
Client Component	A component explicitly sent to the browser because it needs interaction, browser APIs, or client-only libraries.
Server Action	A server function invoked by the application UI to perform a mutation. It must still authenticate, validate, and authorize.
Route Handler	An HTTP endpoint implemented by a `route.ts` file under `app/api/` or another route folder.
RLS	Row Level Security: PostgreSQL rules controlling which rows a database role/user can access.

Term	Plain-language meaning
RPC	Remote Procedure Call: invoking a PostgreSQL function through the Supabase client.
Atomic operation	A database action that succeeds or fails as one indivisible unit, avoiding races between competing requests.
Reservation	Temporary hold on ticket inventory while a buyer completes payment.
Webhook	Server-to-server callback from Paystack announcing a payment event.
Idempotency	Processing the same request/event more than once has no duplicate side effect.
HMAC	A keyed cryptographic signature used here to detect forged or modified ticket tokens and webhooks.
Service-role key	A privileged Supabase credential that bypasses RLS; server-only and highly sensitive.
Generated database types	TypeScript definitions corresponding to the Supabase schema; regenerate/reconcile after schema changes.
Staged image	An initial private upload awaiting server-side validation, conversion, and placement in its final bucket.
Payment provider abstraction	An application contract that keeps business flows independent from a single payment processor.

For maintainers: Keep this guide aligned with the real code. The source copy is `docs/tik-booth-developer-guide.md`; the companion PDF is `docs/Tik-Booth-Developer-Guide.pdf`.